

ESIEA

4A SEPTEMBRE 2022

LE PROJET (QUI REMPLACE L'EXAMEN !)

Cryptologie Appliquée 4A

Enseignants

R.ERRA, S. LARINIER

Semestre S7

2022-2023

Commentaires :

Date limite: à préciser.

- (2) Ce document sera mis à jour à chaque question pertinente qui sera posée.
- (3) Chaque nouvelle version sera "datée" avec la date dans le nom du fichier.



Table des matières

1	Date limite et rendu ?	1
1.1	Date limite	1
1.2	Rendu	1
2	Votre travail	1
2.1	Code python	1
2.2	Projet 1	2
2.3	Quelques questions/réponses	3
3	Introduction	3
4	Contexte	4
4.1	Cycles	4
4.2	Cycles ?	4
5	L’algorithme de recherche de cycle de Floyd	5
5.1	Un exemple	5
6	L’algorithme de recherche de cycle <i>approché</i> de Floyd	5
7	Plus longue sous-chaine commune	6
8	Outils	6
8.1	Notebook Jupyter	6
8.2	Module crypto ?	6
8.3	Module <i>difflib</i> ?	6

1 Date limite et rendu ?

1.1 Date limite

1. On fixera la date limite au **à préciser !**.
2. En cas de changements dans la situation actuelle on avisera.

1.2 Rendu

Vous pourrez déposer votre fichier notebook python sur Moodle, Pas d’emails !

Bon courage à toutes et à tous.

2 Votre travail

2.1 Code python

Vous devez choisir un projet, fixer les différents paramètres et *coder* en python.

2.2 Projet 1

Soit la problématique suivante :

1. Choisir un algorithme $\mathcal{A}$ parmi : DES (mode ECB en priorité mais on peut aussi considérer les autres modes), AES-128 (idem pour le mode), MD5, SHA1, SHA256, etc.
2. Si vous choisissez un algorithme de chiffrement
 - Choisir une valeur initiale x_0 , adaptée à l'algorithme en nombre de bits, aléatoire (ou pas : à vous de choisir).
 - Choisir une valeur de la clé K , adaptée à l'algorithme en nombre de bits, aléatoire (ou pas : à vous de choisir).
3. Si vous choisissez un algorithme de hachage : Choisir une valeur initiale x_0 , adaptée à l'algorithme en nombre de bits, aléatoire (ou pas : à vous de choisir).
4. Considérer l'application $\mathcal{A}$ et rechercher un cycle *approché*.
5. Choisir une distance du style
 - (pour tester votre code) : $\text{distance}(\text{Tortue}, \text{Lievre}) = \text{nombre de bits différents}$
 - puis, dans un second temps :

$$\text{distance}(\text{Tortue}, \text{Lievre}) = \text{nombre de caractères différents}$$

— ou (le plus intéressant)

$$\text{Distance}(\text{Tortue}, \text{Lievre}) = \text{longueur de la plus longue sous-chaîne commune.}$$

6. Et enfin, choisir une valeur de Borne *judicieuse* en fonction de Distance.

Précisions :

1. Le test le plus intéressant à faire, que ce soit avec un algorithme de chiffrement ou un algorithme de hashage est évidemment lorsqu'on utilise $\text{Distance}(\text{Tortue}, \text{Lievre}) = \text{longueur de la plus longue sous-chaîne commune}$ qu'il faut évidemment **normaliser**. En effet, tel quel ce n'est pas une distance, supposons par exemple qu'on travaille avec des mots de 128 bits, soit 16 caractères, il faut donc utiliser :

$$\text{Distance}(\text{Tortue}, \text{Lievre}) = 16 - \text{longueur de la plus longue sous-chaîne commune}$$

Ainsi, on aura donc $\text{Distance}(\text{Tortue}, \text{Tortue}) = 0$.

2. Si vous désirez utiliser

$$\text{distance}(\text{Tortue}, \text{Lievre}) = \text{nombre de bits différents}$$

dans ce cas il **ne faut pas** normaliser, on aura ainsi $\text{Distance}(\text{Tortue}, \text{Tortue}) = 0$ mais je me dois de préciser qu'une haute valeur de cette distance n'est pas très intéressante. Cela devient intéressant pour une chaîne de 128 bits par exemple si on a $\text{Distance}(\text{Tortue}, \text{Tortue}) \leq 64$ (ce qui à mon avis est probablement difficile à atteindre mais sait-on jamais).

3. Qu'est ce qui est vraiment intéressant alors ? Le *graal* serait de trouver un schéma, un *pattern*. Je reprends l'exemple donnée par un étudiant : on choisit $x_0 = \text{"toto"}$ et on calcule un cycle approché pour un algorithme de hachage (donc : pas de clé). Ok, pourquoi pas. Mais alors des questions se posent :
 - Le calcul donne t-il la même chose ou pas pour "toute" chaîne $x_0 = \text{"xyxy"}$, avec x et y deux caractères différents ?
 - Par exemple qu'observe t'on pour la chaîne miroir "otot" ?

- Le calcul donne t-il la même chose ou pas pour "toute" chaîne $x_0 = "xyxyxy"$, avec x et y deux caractères différents ?
 - etc.
 - Et qu'en est-il au niveau des différences entre le code ascii de "t" et celui de "o". En clair, qu'observe t'on pour la chaîne décalée à droite "upup" ? (schéma : $t- > u$ et $o- > p$) ?
 - ou encore qu'observe t-on pour la chaîne décalée à gauche "snsn" ? (schéma : $t- > s$ et $o- > n$) ?
 - etc.
4. Pour celles et ceux qui désirent plutôt étudier un algorithme de chiffrement, on peut se poser le même genre de questions (existence de schémas) sur la chaîne initiale et/ou sur la clé !
 5. À vous de choisir ce que vous voulez mettre en évidence (présence ou absence de schémas) **mais** par contre, je rappelle que ce projet remplace un examen d'1h30, donc je ne vous demande pas de tout chercher, tout tester.

Dans un premier temps soyez disons *prudent.e.s*, si vous cherchez un cycle avec une borne trop précise, cela risque de prendre du temps, voire beaucoup beaucoup de temps.

2.3 Quelques questions/réponses

1. **Q** : À partir de quelle longueur de sous-chaîne commune le "cycle" trouvé est-il pertinent ? **R** : Bonne question mais cela dépend vraiment de l'algorithme choisi (chiffrement ou hachage ?) de l'entropie de votre chaîne initiale ($x_0 = "AAAAAAAAA"$ ou $x_0 = "1z6t9hkl"$?), de l'entropie de la clé etc. Disons que ce n'est pas la qualité de ce que vous trouvez qui m'intéresse mais le fait que votre code puisse, avec du temps, avoir la possibilité de trouver.
2. **Q** : Doit-on prendre plusieurs fonctions de distances possibles ? **R** : Oui, si vous avez le temps, sinon une cela me suffit mais avec plusieurs tests.
3. **Q** : Doit-on prendre tous les algos de hashage ? **R** : Non ! Non ! Non ! Restons *raisonnable*.
4. **Q** : Doit-on prendre tous les cas de bornes possibles ? **R** : Dans l'idéal oui, mais la réalité c'est que vous n'aurez pas le temps ...
5. **Q** : Doit-on essayer Floyd et Floyd approché ? **R** : Si possible oui, bien sûr. Mais un seul des deux est demandé.
6. **Q** : [...] les sous chaînes communes ne sont pas aux mêmes emplacements pour la tortue et le lièvre. Est-ce quand même pertinent ? **R** : Pas grave du tout, c'est même probablement le cas le plus fréquent, par contre, là ça devient **très très très** intéressant de voir si le schéma se répète avec d'autres chaînes similaires dans leur construction (cf remarques sur projet 1 dans la section 2.2).

3 Introduction

- Ce projet remplace l'examen.
- Vous devrez rendre (exclusivement) un notebook jupyter en python 3.
- Le modèle est sur Teams. Merci de **mettre dans le nom du fichier votre nom** svp.

4 Contexte

4.1 Cycles

Soit f une application (non linéaire) d'un ensemble *fini* X (de taille n) dans lui-même : $f : X \rightarrow X$. Si on choisit $x_0 \in X$ et que l'on considère la suite $\{x_0, x_1, x_2 \dots\}$ définie par :

$$x_{i+1} = f(x_i). \quad (1)$$

alors cette suite est *cyclique*. On dit *ultimement cyclique* car au début il y a un "temps" de démarrage pourrait-on dire.

4.2 Cycles ?

Cycle : (Nom commun 1) Du latin *cyclus* emprunté au grec ancien *kyklos* ("cercle, rond, ronde").

Toute suite construite de manière déterministe avec une équation du type (1) sur un ensemble *fini* X , par construction, possède un cycle, *i.e.* il existe un entier n tel que :

$$x_{i+n} = f(x_n). \quad (2)$$

L'image (4.2) donne une illustration graphique de l'existence d'un cycle.

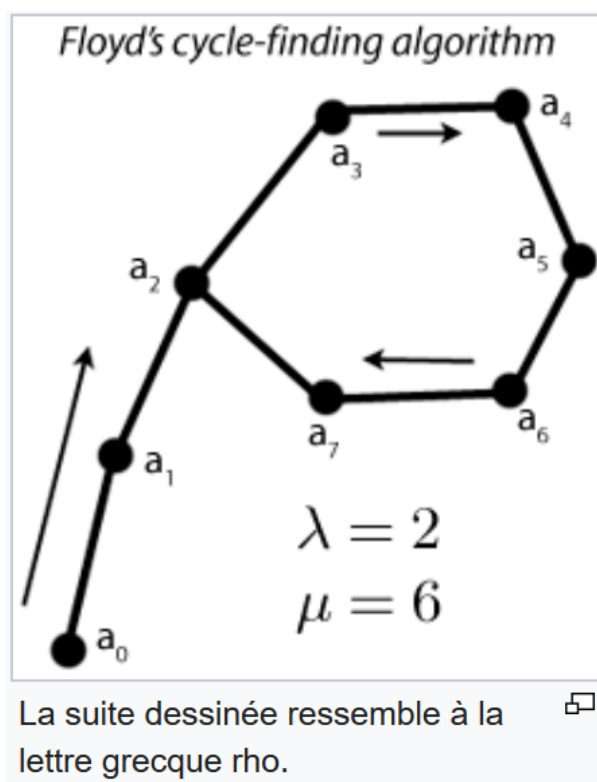


FIGURE 1 – Pseudo-code de l'algorithme de Floyd (Wikipedia)

5 L'algorithme de recherche de cycle de Floyd

Appelé aussi l'*algorithme du Lièvre et de la Tortue*, l'algorithme de détection de cycle de Floyd, est un algorithme pour détecter un cycle (ou disons un multiple de la longueur ce cycle) dans une suite récurrente engendrée par toute fonction f définie d'un ensemble fini X dans lui-même.

```
entrée : une fonction f, une valeur initiale  $a_0$ 
sortie : indice m avec  $a_m = a_{2m}$ 
detection-cycle-floyd(f,  $a_0$ )
    m = 1
    tortue = f( $a_0$ )
    lièvre = f(f( $a_0$ ))

    tant que tortue != lièvre faire
        m = m + 1
        tortue = f(tortue)
        lièvre = f(f(lièvre))

    retourner m
```

FIGURE 2 – Pseudo-code de l'algorithme de Floyd (Wikipedia)

On vous laisse le soin de voir sur Wikipedia (ou autre) la variante de cet algorithme qui, en plus de trouver un multiple de la longueur ce cycle, calcule la longueur exacte de ce cycle.

5.1 Un exemple

Soient $p = 101$, et f l'application de $X = \mathbb{Z}_{101}$ dans lui même ($f : X \rightarrow X$) définie par :

$$f(x) = x^2 + 1 \bmod p \quad (3)$$

On choisit $x_0 = 15 \in X$ et considère la suite $\{x_0, x_1, x_2 \dots\}$ définie par :

$$x_{i+1} = f(x_i) = x_i^2 + 1 \bmod p. \quad (4)$$

On obtient la suite périodique (la période est en gras, indiquée deux fois et séparée par ||) :

$$\{15, 24, \mathbf{72}, \mathbf{34}, \mathbf{46}, \mathbf{97}, \mathbf{17}, \mathbf{88}, \mathbf{69} || \mathbf{15}, \mathbf{24}, \mathbf{72}, \mathbf{34}, \mathbf{46}, \mathbf{97}, \mathbf{17}, \mathbf{88}, \mathbf{69}, 15, 24, 72 \dots\}$$

6 L'algorithme de recherche de cycle *approché* de Floyd

Nous allons nous intéresser à une variante du problème de recherche d'un cycle d'une application. Au lieu de chercher un cycle exact nous allons modifier le critère d'arrêt avec un critère d'*approximation* au lieu d'un critère d'égalité :

```
Tant Que Distance(Tortue, Lievre) <= Borne
    m = m + 1
Tortue = f(Tortue)
Lievre = f(f(Lievre))
Fin du Tant Que
```

où *Distance* donne une mesure de la proximité entre *Tortue* et *lievre* et *Borne* donne une mesure de la précision qu'on cherche.

Donnons un exemple avec la suite définie dans $\mathbb{Z}_{101}$, si on choisit *Borne* = 2 et :

$$\text{Distance}(\textit{Tortue}, \textit{Lievre}) = |\textit{Tortue} - \textit{Lievre}|.$$

On a :

$$\{15, 24, 72, 34, 46, 97, 17, 88 \dots\}$$

alors, 17 vérifie : $\text{Distance}(\textit{Tortue}, \textit{Lievre}) = |15 - 17| \leq 2$.

7 Plus longue sous-chaîne commune

Soient deux chaîne de caractères, par exemple "xyzabcdef" et "abcdefg", la plus *longue sous-chaîne commune* est "**abcdef**" car on ne s'intéresse qu'aux sous-chaînes de caractères consécutives. On peut généraliser le problème en recherchant :

1. Soit la plus *longue sous-chaîne commune* de caractères
2. Soit la plus *longue sous-chaîne commune* de bits.

8 Outils

8.1 Notebook Jupyter

Il est demandé de tout faire dans un notebook.

8.2 Module crypto ?

Il est vivement conseillé de choisir le module *cryptography* qui est très bien documenté. Mais il vous est possible de préférer et donc d'utiliser *cryptdodom*. Cela permettra de faire des comparaisons.

8.3 Module *difflib* ?

Le module *difflib*¹ contient a priori tout ce qu'il vous faut, mais vous pouvez tout recoder si vous le voulez.

1. <https://docs.python.org/3/library/difflib.html>